

Informe: Herramientas open-source y modelos pre-entrenados para análisis de vídeo de fútbol con computer vision (2025-2026)

TL;DR

- **La pila más recomendable y "puntera" para tu herramienta es el ecosistema de Roboflow:** el repo `roboflow/sports` (MIT, ~5.4k estrellas) como pipeline de referencia jugador+balón+campo+radar, `roboflow/trackers` (Apache-2.0) para tracking, `roboflow/supervision` (MIT) para anotación, y detección con **RF-DETR (Apache-2.0, comercial libre)** en lugar de YOLO cuando necesites licencia limpia; para I+D académico el estándar de referencia es **SoccerNet Game State Reconstruction (sn-gamestate)**, que produce directamente el minimapa/radar con dorsales y equipos.
- **La publicación de Camilo Gálvez no pudo verificarse** (LinkedIn bloquea el acceso por robots.txt y no aparece indexada en buscadores); por los hashtags "análisis deportivo / fútbol / machine learning" encaja con el patrón estándar de estos demos: detección YOLO + tracking + clasificación de equipos + homografía a vista cenital. Esto es una inferencia, no un dato confirmado.
- **Aviso legal crítico:** casi todos los demos populares usan **YOLOv8/YOLO11 de Ultralytics, licencia AGPL-3.0**, que obliga a abrir el código de tu producto (incluso si es un servicio web). Para uso comercial cerrado necesitas licencia Enterprise de Ultralytics o sustituir el detector por RF-DETR (Apache-2.0). Los datasets de **SoccerNet están bajo NDA y prohíben el uso comercial**.

Key Findings

1. El repositorio de referencia: `roboflow/sports`

Es el proyecto open-source más maduro y activo para fútbol. Combina cinco componentes en un pipeline con modos `PLAYER_DETECTION`, `PLAYER_TRACKING`, `PITCH_DETECTION`, `TEAM_CLASSIFICATION` y `RADAR`:

- **Detección de jugadores/árbitros/balón:** YOLOv8
- **Detección de campo (32 keypoints):** YOLOv8-pose
- **Tracking:** ByteTrack
- **Clasificación de equipos:** SigLIP (embeddings) + UMAP (reducción de dimensionalidad) + KMeans (clustering en dos equipos)
- **Homografía:** `ViewTransformer` proyecta las posiciones a una vista cenital (radar)

- URL: <https://github.com/roboflow/sports> — Licencia **MIT** — Python — ~5.4k estrellas, ~657 forks
- Ejemplo de fútbol: <https://github.com/roboflow/sports/tree/main/examples/soccer>
- **Nota de licencia del propio README:** el modelo de detección usado en el demo (YOLOv8 de [ultralytics](#)) se distribuye bajo **AGPL-3.0**; solo la capa [sports](#)/[supervision](#) es MIT. Es decir, el repo es MIT pero arrastra AGPL por el detector.

2. Repositorios de tutorial "end-to-end" populares

- **abdullahtarek/football_analysis** — YOLO + KMeans (equipos por color) + optical flow (movimiento de cámara) + transformación de perspectiva; [GitHub](#)
 ↗
 calcula posesión, velocidad y distancia. [GitHub](#)
 ↗
https://github.com/abdullahtarek/football_analysis — 939 estrellas, 324 forks [GitHub](#)
 ↗
 — Licencia MIT — Jupyter/Python. Es el tutorial más seguido y clonado de la comunidad.
- **Hmzbo/Football-Analytics-with-Deep-Learning-and-Computer-Vision** — app Streamlit: detecta jugadores/árbitros/balón, predice equipos por color, construye mapa táctico y rastrea el balón. [GitHub](#)
 ↗
<https://github.com/Hmzbo/Football-Analytics-with-Deep-Learning-and-Computer-Vision> (fork con app web ampliada: <https://github.com/knightstark7/FootBall-Analytics-with-Deep-Learning-and-Computer-Vision>)
- **Tony-Luna/soccer-video-analytics** — YOLOv8 con posesión, heatmaps, homografía a layout cenital, logs CSV/JSON. <https://github.com/Tony-Luna/soccer-video-analytics>
- **AbdelrahmanAtef01/Tactic_Zone** — pipeline ambicioso con detección de tiros/goles/asistencias, velocidad/distancia, sistemas de recomendación de formación e integración de LLM (Gemini). https://github.com/AbdelrahmanAtef01/Tactic_Zone
- **AmmarMohamed0/Football-Analysis-System** — YOLO11 + ByteTrack para detección/tracking en tiempo real. <https://github.com/AmmarMohamed0/Football-Analysis-System>
- **JanSkn/football-computer-vision** — Ultralytics + OpenCV + NumPy, con interpolación del balón y optical flow. <https://github.com/JanSkn/football-computer-vision>

3. Framework de referencia académico (SOTA): SoccerNet Game State Reconstruction

- **SoccerNet/sn-gamestate** (CVPRW'24): construye directamente el minimapa 2D con posición, rol (jugador/portero/árbitro), número de dorsal y equipo (izquierda/derecha) de cada persona, a partir de una sola cámara móvil. Es lo más parecido "de fábrica" a lo que muestran los demos de LinkedIn (radar/bird's-eye view). <https://github.com/SoccerNet/sn-gamestate> — Licencia **GPL-3.0** — Python — 448 estrellas. Componentes del baseline: YOLOv11 (detección), PRTReid/BPBReID (re-ID + equipo + rol), TVCalib/PnLCalib/No-Bells-Just-Whistles (calibración), [GitHub](#)

↗

MMOCR (dorsales).

- Se apoya en **TrackingLaboratory/tracklab**, framework modular de multi-object tracking donde se añaden/reemplazan módulos: <https://github.com/TrackingLaboratory/tracklab>
- Métrica GS-HOTA (tolerancia de localización de 5 m). Paper: <https://arxiv.org/abs/2404.11335> — Vídeo/tutorial: https://www.youtube.com/watch?v=Ir-6D3j_lkA
- Evaluación: fork de TrackEval <https://github.com/SoccerNet/sn-trackeval>

4. Tracking (multi-object tracking)

- **roboflow/trackers** — reimplementaciones limpias de SORT, ByteTrack, OC-SORT, BoT-SORT, C-BIoU y McByte, **licencia Apache-2.0** (apta para producto cerrado, a diferencia de BoxMOT que es AGPL). [GitHub](#)

↗

Se integra con cualquier detector vía `supervision.Detections` (YOLO, DETR, RT-DETR...).

Incluye benchmarks en MOT17, SportsMOT, SoccerNet y DanceTrack.

<https://github.com/roboflow/trackers>

- **ByteTrack** (original, ECCV 2022) — <https://github.com/FoundationVision/ByteTrack> — Licencia MIT
- **BoT-SORT** — añade compensación de movimiento de cámara (clave en TV con paneo/zoom) y re-ID. Según Aharon, Orfaig y Bobrovsky (Tel-Aviv University), "*BoT-SORT: Robust Associations Multi-Pedestrian Tracking*" (arXiv:2206.14651): en MOT17 alcanza **80.5 MOTA, 80.2 IDF1 y 65.0 HOTA**, siendo BoT-SORT-ReID "*the first tracker to achieve IDF1 above 80*"; también primero en MOT20 (77.8 MOTA, 77.5 IDF1, 63.3 HOTA). <https://github.com/NirAharon/BoT-SORT> — MIT

- Ultralytics YOLO ya trae ByteTrack y BoT-SORT integrados [GitHub](#)

↗

(`[bytetrack.yaml, botsort.yaml]`); BoT-SORT es el tracker por defecto.

5. Calibración de cámara / homografía (proyección a campo 2D)

- **mguti97/No-Bells-Just-Whistles** (CVPRW 2024) — registro de campo mediante keypoints con modelo 3D del campo y calibración clásica por DLT; rendimiento superior en calibración 3D en SoccerNet-Calibration, WorldCup 2014 y TS-WorldCup. <https://github.com/mguti97/No-Bells-Just-Whistles>
 - **mguti97/PnLCalib** — extensión con módulo de refinamiento por líneas (Points-and-Lines, optimización no lineal); aceptado en *Computer Vision and Image Understanding*. <https://github.com/mguti97/PnLCalib>
 - **MM4SPA/tvcalib** (TVCalib) — usado como baseline en sn-gamestate. [GitHub](#)
- ↗
- <https://github.com/MM4SPA/tvcalib>
- Alternativa ligera y directa: modelo YOLOv8-pose de Roboflow para 32 keypoints del campo + `[cv2.findHomography]/[ViewTransformer]`. Blog: <https://blog.roboflow.com/camera-calibration-sports-computer-vision/>

- Es el problema más difícil (tamaño pequeño, movimiento rápido, oclusiones). **TrackNet** (heatmap gaussiano sobre múltiples frames consecutivos) es la base clásica. Implementación PyTorch: <https://github.com/yastrebksv/TrackNet> — Paper: <https://arxiv.org/abs/1907.03698>
- Roboflow recomienda **InferenceSlicer / SAHI**: trocear el frame en parches y correr inferencia en cada uno para que el balón sea proporcionalmente mayor; mejora la precisión pero, en palabras del propio blog, *"comes at the cost of reduced speed, as the model needs to run multiple times per frame"*. <https://blog.roboflow.com/tracking-ball-sports-computer-vision/> y <https://blog.roboflow.com/how-to-use-sahi-to-detect-small-objects/>
- Dataset dedicado de balón: <https://universe.roboflow.com/roboflow-jvuqo/football-ball-detection-rejhg>

7. Estimación de pose

- **OpenMMLab MMPose** (RTMPose, ViTPose) es el estándar de la industria. <https://github.com/open-mmlab/mmpose> — Apache-2.0
- **AutoSoccerPose** — análisis 2D/3D de la postura de disparo, con el dataset 3DSP (el mayor dataset anotado de pose 2D de imágenes profesionales de fútbol según los autores). <https://arxiv.org/pdf/2405.12070>
- YOLOv8/YOLO11-pose de Ultralytics para pose rápida integrada en el mismo pipeline de detección.

8. Detección de eventos / action spotting

- **SoccerNet/sn-spotting** — action spotting de 17 clases (goles, tarjetas, cambios) [GitHub](#)
 y ball action spotting de 12 clases (Pass, Drive, Header, High Pass, Out, Cross, Throw In, Shot, Ball Player Block, Player Successful Tackle, Free Kick, Goal).
<https://github.com/SoccerNet/sn-spotting>
- **IRomul/ball-action-spotting** — solución ganadora del reto SoccerNet 2023 (arquitectura slow-fusion 2D+3D CNN; mejora del mAP@1 de ~65% a ~80%).
<https://github.com/IRomul/ball-action-spotting>
- **SoccerNet/sn-teamspotting** — action spotting con predicción del equipo que ejecuta la acción; baseline T-DEED (ganador 2024). <https://github.com/SoccerNet/sn-teamspotting>
- **JeremieOchin/FOOTPASS** — dataset/baseline multimodal player-centric para el reto SoccerNet 2026 (quién hace qué y cuándo). <https://github.com/JeremieOchin/FOOTPASS>
- Para offside/fuera de juego no hay repo open-source robusto "llave en mano"; se construye combinando calibración + tracking + geometría de la línea de defensa (los sistemas profesionales usan multicámara). El fuera de juego automatizado sigue siendo tarea abierta en open-source.

9. Modelos pre-entrenados listos (Roboflow Universe / Hugging Face)

- Detección de jugadores: <https://universe.roboflow.com/roboflow-jvuqo/football-players->

detection-3zvbc

- Detección de balón: <https://universe.roboflow.com/roboflow-jvuqo/football-ball-detection-rejhg>
- Keypoints de campo: <https://universe.roboflow.com/roboflow-jvuqo/football-field-detection-f07vi>
- Copia del modelo de campo (YOLOv8-pose) en Hugging Face: <https://huggingface.co/Simon9/football-field-detection-roboflow>
- **RF-DETR** (detector base, ver §Details): <https://github.com/roboflow/rf-detr>

10. Datasets abiertos para fine-tuning

- **SoccerNet** — Según Giancola et al., "SoccerNet: A Scalable Dataset for Action Spotting in Soccer Videos" (CVPR 2018 Workshops), "*the dataset is composed of 500 complete soccer games from six main European leagues, covering three seasons from 2014 to 2017 and a total duration of 764 hours*", con 6.637 anotaciones temporales y densidad media de un evento cada 6,9 min (split 300/100/100). Cubre tracking, calibración, re-ID, action spotting y GSR. <https://www.soccer-net.org/data> — **bajo NDA, solo investigación no comercial.**
- **DFL Bundesliga Data Shootout** (Kaggle, ~400 imágenes anotadas de ball/portero/jugador/árbitro; también clips completos): <https://www.kaggle.com/competitions/dfl-bundesliga-data-shootout> — es el dataset base del tutorial de Roboflow.
- **SoccerTrack v2** (vídeo de **cámara fija**, reto MMSports 2025) — especialmente relevante si tu entrada no es de TV: <https://sites.google.com/g.sp.m.is.nagoya-u.ac.jp/stc2025>
- Datasets de Roboflow Universe (varios bajo CC BY 4.0), p.ej. <https://universe.roboflow.com/soccernet-qkw5c/soccernet-vmss0>

Details

Comparativa de las opciones "punteras"

- **Máxima facilidad de implementación:** [abdullahtarek/football_analysis](#) o [roboflow/sports](#) — ambos con notebooks de Colab, pesos descargables y documentación abundante. Permiten un radar/heatmap funcional en horas.
- **Máxima precisión y completitud "de fábrica" (minimapa con dorsales y equipos):** [SoccerNet/sn-gamestate](#) sobre TrackLab. Más pesado de instalar (uv/conda, msvc 2.0.1, Hydra) pero es el estado del arte reproducible.
- **Detector recomendado para producto comercial: RF-DETR** de Roboflow. Según la documentación oficial (rfdetr.roboflow.com), RF-DETR-L alcanza 56.5 AP50:95 en COCO a 6.8 ms (NVIDIA T4, TensorRT FP16) y **RF-DETR-2XL logra 60.1 AP50:95, el primer modelo en tiempo real que supera 60 AP en COCO**. Backbone DINOv2, familia de 6 tamaños (Nano 30.5M params/384×384 → 2XL 126.9M/880×880), primera publicación el 20 de marzo de 2025, aceptado en ICLR 2026 (autores I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, N. Peri). **Matiz de licencia:** Nano→Large son Apache-2.0 (comercial libre, sin copyleft); **XL/2XL requieren [rfdetr\[plus\]](#) bajo licencia PML 1.0, no Apache-2.0.** <https://github.com/roboflow/rf-detr>

- **Tracking en producto comercial:** `roboflow/trackers` (Apache-2.0) con BoT-SORT (compensa el paneo/zoom de la cámara de TV).
- **Anotación/visualización/post-proceso:** `roboflow/supervision` (MIT), model-agnostic, con anotadores, zonas, tracking y métricas. <https://github.com/roboflow/supervision>

Flujo completo típico (de vídeo crudo a radar)

1. Extraer frames → 2. Detectar jugadores/árbitros/balón (RF-DETR o YOLO) → 3. Tracking (ByteTrack/BoT-SORT) → 4. Clasificar equipos (SigLIP+UMAP+KMeans, o por color HSV) → 5. Detectar 32 keypoints del campo → 6. Calcular homografía → 7. Proyectar posiciones a plano 2D → 8. Generar radar, heatmaps, diagramas de Voronoi (control de territorio), posesión, velocidad/distancia, sprints. El tutorial de Roboflow "Football AI: From Basics to Advanced Stats with Python" cubre exactamente esta cadena: vídeo <https://www.youtube.com/watch?v=Xwou5qO--vY>, notebook en <https://github.com/roboflow/notebooks> (how-to-track-football-players.ipynb).

Requisitos de hardware y velocidad

- **Se requiere GPU CUDA.** Los tutoriales de Roboflow están diseñados para una GPU **T4 gratuita de Google Colab**.
- Una reproducción independiente del ejemplo de fútbol de Roboflow reporta que corre en una **GeForce RTX 3090 usando ~3 GB de VRAM**; el entrenamiento de los modelos tomó ~1 h (keypoints, T4), ~40 min (jugadores, L4) y ~1,3 h (balón, A100).
- **Throughput de YOLO11 en T4 (TensorRT FP16, 640 px, batch 1)**, según la documentación oficial de Ultralytics: n=1.5 ms (~667 FPS), s=2.5 ms (~400 FPS), m=4.7 ms (~213 FPS), l=6.2 ms (~161 FPS), x=11.3 ms (~88 FPS). Son latencias de inferencia pura, sin tracking ni post-proceso.
- Un mantenedor de Ultralytics indica que una **RTX 4090 supera cómodamente los 30 FPS** incluso con resoluciones altas, mientras que una GTX 1650 solo logra "2-4 FPS en vídeo 720p" con YOLOv11s.
- Pipelines pesados completos (varios modelos por frame, p.ej. con SAM2 o multi-etapa) rinden **~1-2 FPS en T4**.
- El **sn-gamestate ejecuta 9 etapas secuenciales** y está lejos de tiempo real; se ajusta el `batch_size` para evitar quedarse sin memoria. No hay una cifra oficial de FPS publicada (dato inferido de la arquitectura, no citado).
- El tracking del balón con troceado (SAHI/InferenceSlicer) multiplica las inferencias por frame y reduce mucho la velocidad; el coste de subir de 640 a 1280 px crece de forma marcada.

Sobre la publicación de Camilo Gálvez

No fue posible acceder a la publicación (LinkedIn bloquea el scraping por robots.txt) ni localizarla vía buscadores. Por el slug y los hashtags ("análisis deportivo", "fútbol", "machine learning"), el contenido encaja con el patrón de estos proyectos: tracking de jugadores y balón + estadísticas + mapa táctico. **No se puede confirmar qué stack concreto usó;** lo

más probable, dado el ecosistema dominante en la comunidad hispana de análisis deportivo, es alguna variante de YOLO + ByteTrack + homografía (estilo `roboflow/sports` o `abdullahtarek/football_analysis`). Trátase como hipótesis, no como hecho.

Recommendations

Fase 1 — Prototipo rápido (días): Clona `roboflow/sports` (MIT) y ejecuta el ejemplo de fútbol en modo `RADAR` con los modelos pre-entrenados de Roboflow Universe. Valida sobre tu vídeo de buena calidad. Añade `supervision` (MIT) para anotación y `roboflow/trackers` para el tracking. Alternativa aún más didáctica: `abdullahtarek/football_analysis`.

Fase 2 — Precisión (semanas): Si necesitas dorsales, equipos y minimapa robustos, monta `SoccerNet/sn-gamestate` sobre TrackLab. Haz fine-tuning del detector con el dataset DFL de Kaggle (permisivo) y/o tus propios clips anotados en Roboflow. Para calibración, prueba PnLCalib y No-Bells-Just-Whistles y quédate con el que mejor tolere el paneo de tu cámara.

Fase 3 — Producto comercial: Sustituye YOLO por **RF-DETR tamaño Nano→Large (Apache-2.0)** como detector y usa `roboflow/trackers` (Apache-2.0) con BoT-SORT. Así eliminas la obligación copyleft de AGPL sin pagar licencia. Evita XL/2XL si quieres mantenerte fuera de la licencia PML 1.0. Verifica la licencia de cada módulo de calibración/pose antes de integrarlo.

Umbralles que cambian la decisión:

- Si el vídeo es de **cámara fija** (no TV), prioriza SoccerTrack v2 y modelos entrenados para vista amplia/cenital; la homografía es más estable pero los jugadores aparecen más pequeños.
- Si necesitas **tiempo real (>25 FPS)**, usa RF-DETR/YOLO nano/small + TensorRT en GPU ≥ RTX 4090, y reserva SAHI solo para el balón.
- Si el **balón es el objetivo crítico**, presupuesta la penalización de velocidad del troceado y considera un modelo dedicado (TrackNet o el dataset de balón de Roboflow).

Caveats

- **Licencia YOLO (Ultralytics) = AGPL-3.0:** cubre código Y pesos entrenados, y se activa incluso en despliegue como servicio web (Sección 13 de la AGPL). Para producto cerrado necesitas la Enterprise License de Ultralytics o cambiar a RF-DETR. Muchos repos de tutorial etiquetados "MIT" dependen internamente de Ultralytics, heredando la restricción AGPL en la práctica (el propio README de `roboflow/sports` lo advierte).
- **RF-DETR:** Nano→Large son Apache-2.0 (libres para comercial); XL/2XL van bajo PML 1.0 vía `rfdetr[plus]`. No asumas Apache-2.0 para toda la familia.
- **Datasets SoccerNet bajo NDA:** prohibida la redistribución de los vídeos y el uso comercial; solo investigación. Las anotaciones/metadatos pueden ser MIT/CC BY, pero los vídeos no.
- No se pudo verificar el contenido exacto del post de LinkedIn de Camilo Gálvez; las inferencias sobre su stack son probabilísticas, no confirmadas.

- Las cifras de FPS son latencia de inferencia con TensorRT en condiciones ideales; el rendimiento real end-to-end (con tracking, homografía y detección de balón por troceado) es sustancialmente menor.
- El número de estrellas y el estado de mantenimiento de los repos cambian con el tiempo; verifica la actividad reciente (commits) antes de comprometerte con uno.